09 | 聪明的改进 II：字面量、静态断言和成员函数初探

Tony Bai，资深后端开发，关注 Go、云原生、区块链、人工智能等领域的技术实践与趋势。

2025-07-21 08:00 北京

限时 7 折，365 天阅读权限仅需 69 元
推荐 Kindle 用户直接在亚马逊下单，Kindle 会自动推送到设备上，无需手动传输。
推荐 Kindle 用户直接在亚马逊下单，Kindle 会自动推送到设备上，无需手动传输。
推荐 Kindle 用户直接在亚马逊下单，Kindle 会自动推送到设备上，无需手动传输。

Tony 大厂，资深后端开发，关注 Go、云原生、区块链、人工智能等领域的技术实践与趋势。

- 聪明的改进：字面量、静态断言
- 成员函数：初探
- 今天，我会教你字面量、静态断言和成员函数。
- 上周，我们聪明的改进：函数和变量。
- 以 Python 为例，我们为你展示了函数、变量、类型定义和 import 语句。

```
# hello.py
def greeting(name):
    message = f"Hello, {name}!"
    return message

print("Starting greetings...")

greeting_1 = greeting("Alice")
print(greeting_1)

greeting_2 = greeting("Bob")
print(greeting_2)
```

输出结果：

```
$ python hello.py
Starting greetings...
Hello, Alice!
Hello, Bob!
```

上周这个例子展示了 Python 语言的灵活性和简洁性。对于脚本语言，Python 的优势在于：

- 无需显式类型声明：Python 会在运行时自动推断变量的类型。
- 动态执行：Python 的解释器会逐行执行代码，并动态地创建对象和分配内存。

这种灵活性和简洁性使得脚本编写变得极其高效，特别适合快速原型设计和日常任务。

```
# hello.go
package main

import "fmt"

func greeting(name string) string {
    message := fmt.Sprintf("Hello, %s!", name)
    return message
}

func main() {
    fmt.Printf("Starting greetings...\n")

    greeting_1 := greeting("Alice")
    fmt.Printf("%s\n", greeting_1)

    greeting_2 := greeting("Bob")
    fmt.Printf("%s\n", greeting_2)
}
```

编译并运行这个程序，我们可以看到它的输出结果：

```
$ go build hello.go
$ ./hello
Starting greetings...
Hello, Alice!
Hello, Bob!
```

编译器会自动推断出字符串变量的类型。

可是，思考一下，这个 Python 程序的 import 语句呢？

```
import math

print("Starting greetings...")

greeting_1 = greeting("Alice")
print(greeting_1)

greeting_2 = greeting("Bob")
print(greeting_2)
```

如果直接编译这个程序，会发生什么？

```
$ go build hello.go
# command-line-arguments
./hello.go:3:8: imported and not used: "math" as math
```

编译器会报告说，导入的 math 模块从未被使用。

这种严格的检查，在大型项目中，能有效地防止因拼写错误或遗忘清理导入而引入的潜在问题。

字面量

- 字面量（Literal）：在代码中，直接写入一个固定值的语法。
- 为什么需要字面量？
- 提高可读性：让代码更清晰、更易于理解。
- 减少错误：避免手动计算或输入可能出错的值。
- 编译器优化：编译器可以对字面量进行一些优化，例如内联（Inlining）。
- 字面量的分类：
- 布尔字面量：true 和 false。
- 整数字面量：10，0x1A（十六进制），0b1010（二进制）。
- 浮点数字面量：3.14，1.0e-9。
- 字符串字面量："hello"，`raw string literal`。
- 复合字面量：[1, 2, 3]，{"key": "value"}，MyStruct{field: 10}。
- 字面量的类型推断：编译器会根据上下文，为字面量推断出合适的类型。
- 例如，在 var x int = 10 中，10 被推断为 int 类型。
- 在 var y float64 = 10 中，10 被推断为 float64 类型。
- 字面量的不可变性：字面量通常是不可变的。
- 例如，你不能修改 10 这个数字本身。
- 字面量的内存分配：字面量的值通常会被编译器放入只读内存区域。

静态断言

- 断言（Assertion）：一个关于程序状态的声明，通常用于调试和测试。
- 静态断言：在编译时（Compile Time）进行的断言。
- 为什么需要静态断言？
- 早期发现错误：在代码运行之前，就发现潜在的错误。
- 提高代码质量：确保代码满足某些特定的约束条件。
- 文档作用：断言本身也是一种对代码意图的文档。
- 静态断言的实现方式：
- 编译器内建支持：编译器会内建对静态断言的支持。
- 宏（Macro）：通过宏来实现静态断言。
- 模板（Template）：利用模板来实现静态断言。
- 静态断言的应用场景：
- 检查类型大小：确保某个类型的大小符合预期。
- 检查枚举值：确保枚举值的数量或范围符合预期。
- 检查配置：确保编译时配置的组合是有效的。
- 检查接口实现：确保某个类型实现了某个接口的所有方法。

成员函数

- 成员函数（Member Function）：在类（Class）或结构体（Struct）内部定义的函数。
- 为什么需要成员函数？
- 封装性：将数据和操作数据的函数放在一起，隐藏内部实现细节。
- 可读性：让代码更符合面向对象的思维方式，更容易理解和维护。
- 多态性：为不同的类型提供相同的接口，实现不同的行为。
- 成员函数的语法：
- 在类/结构体内部定义：通常在类/结构体的内部定义成员函数。
- 访问成员变量：成员函数可以访问类/结构体的成员变量。
- 隐式参数：成员函数通常会有一个隐式参数，指向调用该函数的对象。
- 静态成员函数：属于类/结构体本身，而不是属于某个对象的函数。
- 虚函数（Virtual Function）：可被子类重写（Override）的成员函数。
- 纯虚函数（Pure Virtual Function）：没有实现的虚函数，通常用于定义接口。
- 成员函数的实现：
- 内联（Inline）：编译器可以将成员函数的调用直接替换为函数的实现，以减少函数调用的开销。
- 虚函数表（Vtable）：用于实现多态性的数据结构。
- 成员函数的调用：
- 通过对象调用：obj.member_function()。
- 通过指针调用：ptr->member_function()。
- 静态成员函数调用：ClassName::static_member_function()。

今天的作业

- 字面量：
- 为你的 Python 程序，添加一个布尔字面量，用于控制是否输出某条信息。
- 为你的 Python 程序，添加一个浮点数字面量，用于表示一个圆的半径，并计算其面积。
- 为你的 Go 程序，添加一个整数字面量，用于表示一个端口号。
- 为你的 Go 程序，添加一个字符串字面量，用于表示一个欢迎语。
- 为你的 Go 程序，添加一个复合字面量，用于表示一个坐标点 {x, y}。
- 静态断言：
- 为你的 Go 程序，编写一个宏，用于检查一个整数是否大于 0。
- 为你的 Go 程序，编写一个宏，用于检查一个字符串是否非空。
- 成员函数：
- 为你的 Python 程序，定义一个简单的类，并为其添加一个成员函数。
- 为你的 Go 程序，定义一个结构体，并为其添加一个成员函数。

今天的思考

- 字面量的类型推断，在不同的语言中，有哪些不同？
- 静态断言，在哪些情况下，会比运行时断言更有用？
- 成员函数，是面向对象编程的核心吗？

参考资料

- Python 官方文档
- Go 官方文档
- 《Effective Go》
- 《Clean Code》

下期预告

- 聪明的改进 III：函数、变量、类型定义和 import 语句。
- 为你的 Python 程序，添加一个函数。
- 为你的 Go 程序，添加一个变量。
- 为你的 Python 程序，定义一个简单的类型。
- 为你的 Go 程序，定义一个简单的类型。
- 为你的 Python 程序，添加一个 import 语句。
- 为你的 Go 程序，添加一个 import 语句。

课程学习打卡
关注极客时间服务号
每日学习签到
极客时间

分享 收藏 赞 0 评论 0 复制链接 分享给好友 生成海报 分享给朋友 收藏 赞 0 评论 0 复制链接 分享给好友 生成海报 分享给朋友

发表于 1 天前 发表于 1 天前 发表于 1 天前